

Chrono Rift

Docker and Build Setup Guide

1. Why Docker

Chrono Rift uses POSIX shared memory, unnamed semaphores, signals (SIGUSR1, SIGALRM), and pthreads — all Linux-only system calls. Docker ensures every student compiles and runs inside the same Ubuntu 22.04 environment, eliminating dependency issues and guaranteeing consistent grading.

All three processes — arbiter, hip, and asp — run inside the same container and share a Linux namespace. Shared memory and signals work without any special Docker flags.

Note: Use the ncurses (TUI) interface on macOS. SFML GUI is unreliable inside Docker on macOS due to OpenGL conflicts.

2. Install Docker

Download Docker Desktop from: <https://www.docker.com/products/docker-desktop>

OS	Action Required
Linux	Install Docker Desktop, then run: <code>sudo usermod -aG docker \$USER</code> and log out and back in.
macOS	Docker Desktop installs and runs without extra configuration.

3. Required Folder Structure

Your final submission zip must follow the structure below exactly. Each process must have its own folder with a designated main entry point. You may add as many additional .h and .cpp files as needed for code organisation.

```

submission/
├── Dockerfile           # include unchanged
├── Makefile             # uncomment LIBS line for your GUI choice
├── requirements.txt     # list any extra apt packages (leave blank if none)
├── arbiter/
│   ├── arbiter.cpp     # main entry point — must produce the 'arbiter'
│   └── executable
│       ├── *.h         # optional helper headers
│       └── *.cpp       # optional helper source files
├── hip/
│   ├── hip.cpp         # main entry point — must produce the 'hip' executable
│   ├── *.h             # optional helper headers
│   └── *.cpp           # optional helper source files
├── asp/
│   ├── asp.cpp         # main entry point — must produce the 'asp' executable
│   ├── *.h             # optional helper headers
│   └── *.cpp           # optional helper source files
└── report.pdf          # turnaround analysis and screenshots

```

Important: Each process must produce exactly one executable (arbitrator, hip, asp). You are free to split your implementation across as many .h/.cpp files as you like within each process folder.

4. requirements.txt

As a best practice in containerised development, package dependencies are declared separately from the Dockerfile. The requirements.txt file lists any additional apt packages your code needs beyond what is already installed in the Dockerfile.

The following packages are already provided by the Dockerfile — do not repeat them:

```
build-essential      cmake      gdb
libsFML-dev          libSDL2-dev  libglfw3-dev  libncurses-dev
```

Format — one package name per line:

```
# requirements.txt
# List any EXTRA apt packages your code needs beyond the Dockerfile defaults.
# Leave blank (or with comments only) if nothing extra is required.
#
# Example:
# libboost-all-dev
# libjpeg-dev
```

Important: Leave requirements.txt blank if your code compiles with the default Dockerfile. Only list packages not already in the Dockerfile.

5. Dockerfile

Copy this file as-is into your submission folder. The filename must be Dockerfile with no extension. It reads requirements.txt at build time and installs any extra packages you have listed, on top of the standard build tools and GUI libraries.

```
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

# Install core build tools and all supported GUI libraries
RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
    gdb \
    libsFML-dev \
    libSDL2-dev \
    libglfw3-dev \
    libncurses-dev \
    && rm -rf /var/lib/apt/lists/*

# Copy and install any extra packages declared in requirements.txt
COPY requirements.txt /tmp/requirements.txt
RUN grep -v '^#' /tmp/requirements.txt | grep -v '^$' | \
```

```
xargs -r apt-get install -y && rm -rf /var/lib/apt/lists/*

WORKDIR /app

CMD ["bash"]
```

Build the image once from inside your submission folder:

```
docker build -t chrono-rift-env .
```

Note: This takes 2–5 minutes the first time. Repeat only if you modify the Dockerfile or requirements.txt.

6. Makefile

Copy this Makefile into your submission folder. Uncomment the one LIBS line that matches your GUI library. Do not change anything else.

```
CXX      = g++
CXXFLAGS = -Wall -Wextra -std=c++17 -pthread

# Uncomment the one LIBS line that matches your GUI choice:
LIBS = -lsfml-graphics -lsfml-window -lsfml-audio -lsfml-network -lsfml-system
-lrt
# LIBS = $(shell sdl2-config --libs) -lrt
# LIBS = -lglfw -lGL -lrt
# LIBS = -lcurses -lrt

TARGETS = arbiter hip asp

all: clean $(TARGETS)
    @echo Build complete.

arbiter: arbiter/arbiter.cpp
    $(CXX) $(CXXFLAGS) arbiter/*.cpp -o $@ $(LIBS)

hip: hip/hip.cpp
    $(CXX) $(CXXFLAGS) hip/*.cpp -o $@ $(LIBS)

asp: asp/asp.cpp
    $(CXX) $(CXXFLAGS) asp/*.cpp -o $@ $(LIBS)

clean:
    rm -f $(TARGETS)

.PHONY: all clean
```

Critical: The indentation before each compile command must be a tab character, not spaces. If copied incorrectly, make will fail with 'missing separator'.

7. Running the Container

Linux

```
# Run once per terminal session
xhost +local:docker

# Run every time you start the container
docker run -it --rm \
  -e DISPLAY=$DISPLAY \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  -v $(pwd):/app \
  chrono-rift-env
```

macOS (ncurses — recommended)

```
docker run -it --rm \
  -v $(pwd):/app \
  chrono-rift-env
```

Note: No display flags are needed for ncurses. If you need SFML on macOS, install XQuartz and add: `-e DISPLAY=host.docker.internal:0` to the run command.

8. Daily Development Workflow

Once inside the container terminal, your standard workflow is:

```
# Build all three processes
make

# Run all three in background
./arbiter & ./hip & ./asp &

# Stop all background processes
kill %1 %2 %3

# Clean and rebuild
make clean && make
```

Tip: Your source files are live-synced via the volume mount. Edit on your host machine and rebuild inside the container without restarting it.

9. Submission Checklist

File / Folder	Requirement
Dockerfile	Required — include unchanged
Makefile	Required — must compile all three targets

File / Folder	Requirement
requirements.txt	Required — list any additional apt packages beyond Dockerfile defaults. Leave blank if none.
arbiter/	Required — must contain arbiter.cpp as main entry point. Additional .h and .cpp files allowed.
hip/	Required — must contain hip.cpp as main entry point. Additional .h and .cpp files allowed.
asp/	Required — must contain asp.cpp as main entry point. Additional .h and .cpp files allowed.
report.pdf	Required — turnaround analysis and screenshots
Pre-compiled binaries	NOT ALLOWED — will be deleted before grading

10. Common Errors

Error	Fix
Cannot connect to X server	Run <code>xhost +local:docker</code> on the host before starting the container
missing separator (Makefile)	Tab characters were replaced with spaces — re-copy the Makefile
SFML/Graphics.hpp not found	Image was built before Dockerfile was saved — re-run <code>docker build</code>
Segfault on <code>shm_open</code>	Stale shared memory from a previous run — call <code>shm_unlink</code> at startup
requirements.txt packages not found	Check spelling of package names — run <code>apt-cache search <name></code> inside the container to verify